GARAGE MANAGEMENT CRM - COMPLETE TECHNICAL DOCUMENTATION

Version: 1.0
Date: July 2026
Project: Garage Management CRM for Hostinger Deployment
Development Environment: Google Antigravity
Target Platform: Hostinger Cloud Startup Plan

TABLE OF CONTENTS

1. Project Overview
2. System Architecture
3. Technical Stack
4. User Roles & Permissions
5. Database Schema
6. Folder Structure
7. Module Specifications
8. API Endpoints
9. WhatsApp Integration
10. Backup & Restore System
11. Security Implementation
12. Local Development Setup
13. Hostinger Deployment Guide
14. Testing Checklist
15. Maintenance & Monitoring
16. Troubleshooting Guide

---

1. PROJECT OVERVIEW

1.1 Introduction

The Garage Management CRM is a comprehensive web-based solution designed specifically for Indian auto garages to digitize their entire workflow. This system replaces manual registers and sticky notes with a modern, efficient digital platform accessible from anywhere.

1.2 Key Features

- ✅ Digital Job Cards with photo documentation
- ✅ Customer & Vehicle Management (supports multiple vehicles per customer)
- ✅ Real-time Inventory Management with Barcode Scanner
- ✅ Financial Tracking & Invoicing (GST compliant)
- ✅ Automated WhatsApp Status Updates
- ✅ Mobile Responsive Design (works on all devices)
- ✅ Role-based Dashboards (Owner, Mechanic, Customer)
- ✅ Complete Backup & Restore System
- ✅ Appointment Management
- ✅ Multi-mechanic assignment
- ✅ Before/After/Issue photo documentation

1.3 Business Benefits

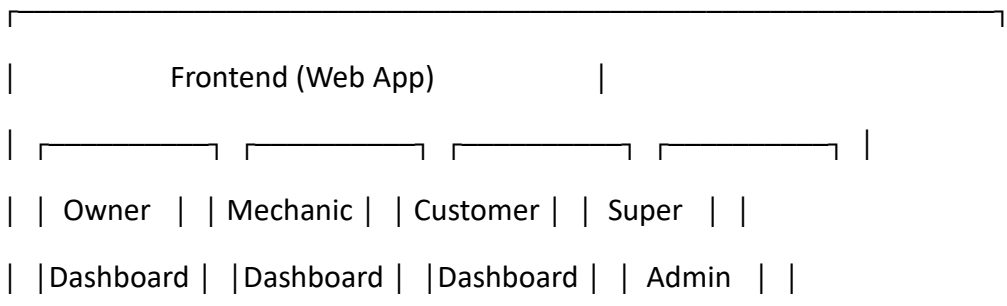
- 📈 80% reduction in job card creation time
- 💰 Faster payment recovery through automated reminders
- 🛡️ Zero dispute claims through photo documentation
- 😊 Improved customer satisfaction via real-time updates
- 📊 Complete digital history for better business decisions

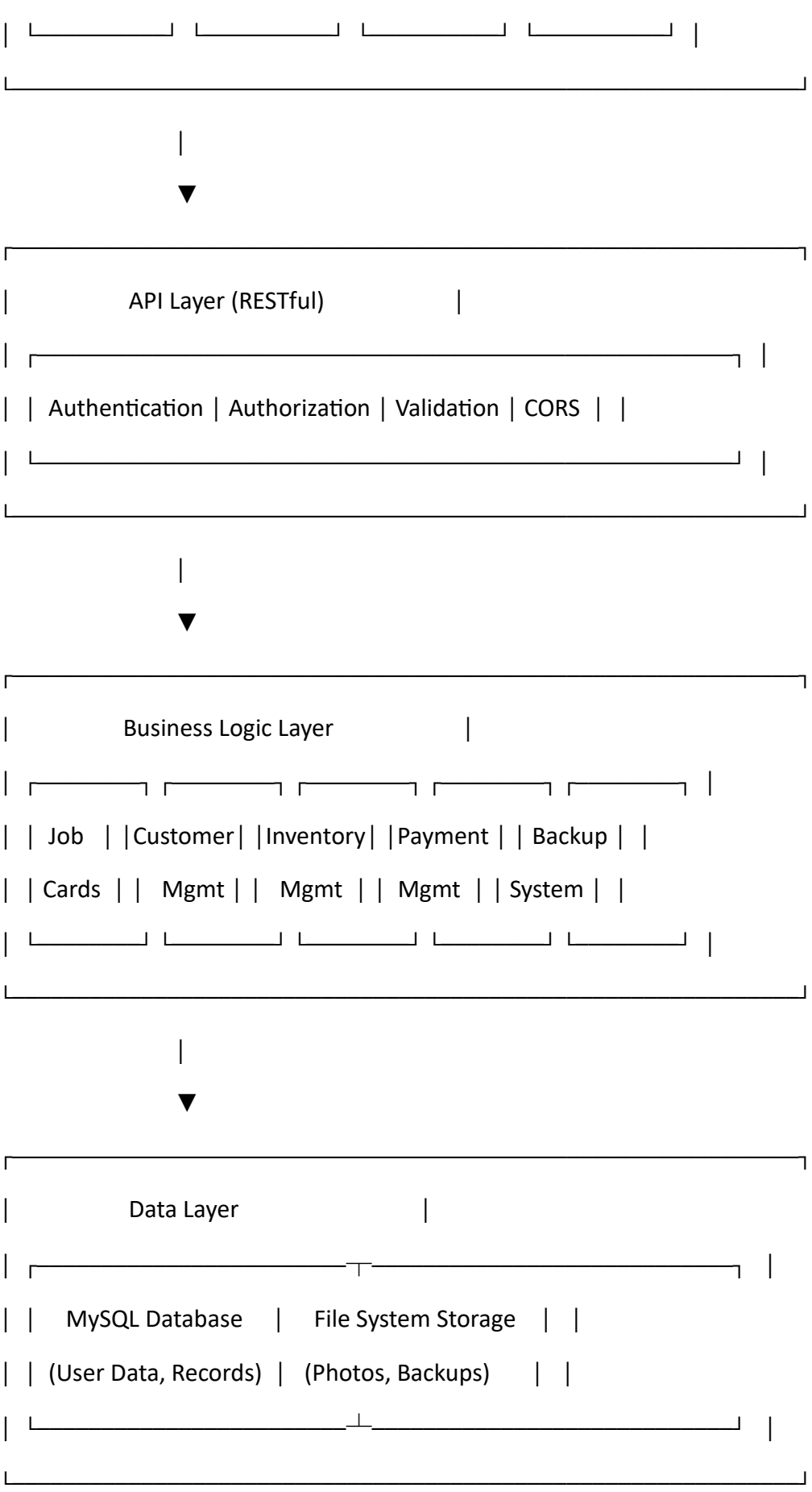
---

2. SYSTEM ARCHITECTURE

2.1 Architecture Diagram

text




API Layer (RESTful)
Authentication | Authorization | Validation | CORS
Business Logic Layer
Job Cards
Customer Mgmt
Inventory Mgmt
Payment Mgmt
Backup System
Data Layer
MySQL Database
(User Data, Records)
File System Storage
(Photos, Backups)

2.2 Data Flow

1. **User Action** → Browser sends request
2. **API Layer** → Authenticates and validates
3. **Business Logic** → Processes request
4. **Data Layer** → Reads/Writes to database or storage
5. **Response** → Returns JSON data to frontend
6. **UI Update** → Renders updated interface

---

3. TECHNICAL STACK

3.1 Backend Technologies

Component	Technology	Version	Purpose
Framework	Laravel/PHP	8.0+	Core application development
Database	MySQL	5.7+	Data storage
Cache	Redis (optional)	Latest	Performance optimization
Queue	Database/Redis	-	Background jobs (WhatsApp)

3.2 Frontend Technologies

Component	Technology	Version	Purpose
Framework	React/Vue.js	Latest	Interactive UI
CSS Framework	Bootstrap/Tailwind	Latest	Responsive design
JavaScript	ES6+	-	Client-side logic
Barcode Scanner	QuaggaJS	Latest	Barcode scanning

3.3 Third-Party Integrations

Service	Purpose	Notes
WhatsApp Business API	Automated notifications	For status updates, reminders
Email Service	Email notifications	For invoices, estimates

3.4 Development Tools

Tool	Purpose	Notes
Google Antigravity	Development Environment	Primary IDE
Git	Version Control	For code management
Composer	PHP Dependency Manager	-
NPM/Yarn	JavaScript Package Manager	-

---

4. USER ROLES & PERMISSIONS

4.1 Role Definitions

🔐 Super Admin (Developer)

- **Access Level:** Full system access
- **Permissions:**
  - Manage all users and roles
  - View all modules and data
  - Backup and restore system
  - System configuration
  - Module visibility control

👔 Owner

- **Access Level:** Complete business management
- **Permissions:**

    - Financial overview and reports
    - Create and manage job cards
    - Manage inventory and suppliers
    - Customer and vehicle management
    - Manage mechanics (attendance, payroll)
    - Module visibility settings
    - Send WhatsApp communications

🔧 **Mechanic**

- **Access Level:** Workshop operations only
- **Permissions:**
    - Create job cards (auto-assigned to them)
    - View assigned job cards
    - Update job status
    - Upload photos (before/after/issue)
    - Mark attendance
    - Apply for leave
    - **No access to:** Financial data, inventory management

👤 **Customer**

- **Access Level:** Own vehicles and services only
- **Permissions:**
    - View own vehicles
    - Create service appointments
    - Track job status
    - View service history
    - Accept/reject repair recommendations
    - Update profile and contact info
    - View invoices and payment history

4.2 Permission Matrix

Module	Super Admin	Owner	Mechanic	Customer
Dashboard	✅ Full	✅ Full	⚠️ Limited	⚠️ Limited
Job Cards	✅ Full	✅ Full	✅ Create/Update	👁️ View Own
Inventory	✅ Full	✅ Full	👁️ View Only	❌ No
Payments	✅ Full	✅ Full	❌ No	👁️ View Own
Customers	✅ Full	✅ Full	👁️ View	👤 Own Only
Vehicles	✅ Full	✅ Full	👁️ View	👤 Own Only
Reports	✅ Full	✅ Full	❌ No	❌ No
Settings	✅ Full	✅ Full	❌ No	❌ No
Backup	✅ Full	❌ No	❌ No	❌ No
Appointments	✅ Full	✅ Full	👁️ View	✅ Create
Attendance	✅ Full	✅ Full	✅ Own	❌ No

Legend:

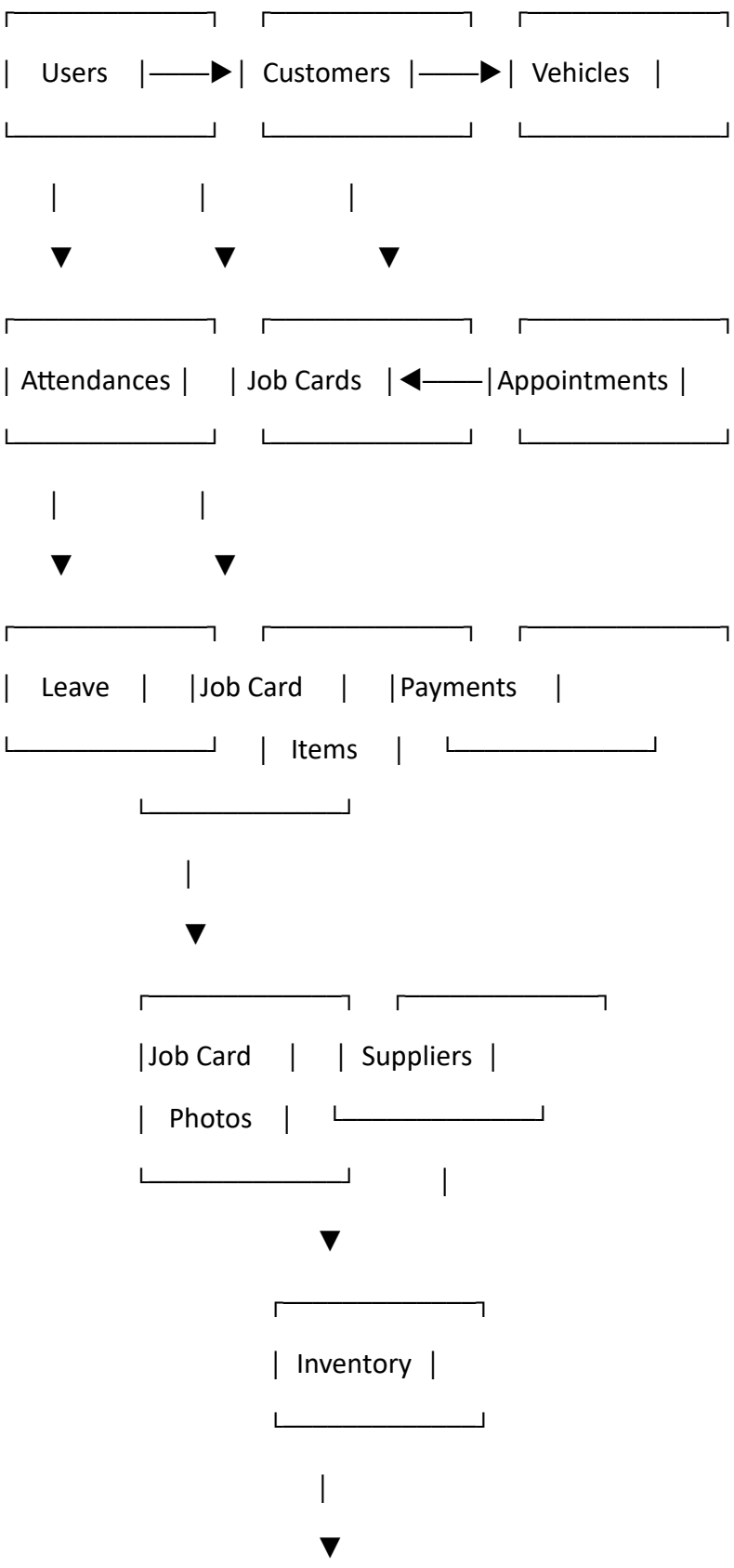
- ✅ Full = Complete access (Create, Read, Update, Delete)
- ⚠️ Limited = Partial access
- 👁️ View Only = Read-only access
- 👤 Own Only = Access to own records only
- ❌ No = No access

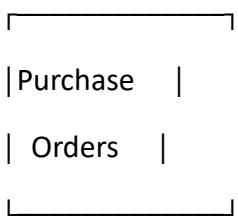
---

5. DATABASE SCHEMA

5.1 Entity Relationship Diagram (ERD)

text





5.2 Complete SQL Schema

sql

```
-- ============================================
-- GARAGE MANAGEMENT CRM DATABASE SCHEMA
-- Version: 1.0
-- ============================================


-- ------------------------------------------------------
-- Table: users
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(100) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    role ENUM('super_admin', 'owner', 'mechanic', 'customer') NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    profile_image VARCHAR(500) DEFAULT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    last_login TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

```
    INDEX idx_role (role),
    INDEX idx_phone (phone)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: customers (extends users with role='customer')
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS customers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT UNIQUE,
    address TEXT,
    gst_number VARCHAR(50) DEFAULT NULL,
    whatsapp_number VARCHAR(20),
    preferred_contact ENUM('phone', 'whatsapp', 'email') DEFAULT 'whatsapp',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_whatsapp (whatsapp_number)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: vehicles
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS vehicles (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    registration_number VARCHAR(20) UNIQUE NOT NULL,
```

```
    chassis_number VARCHAR(50) DEFAULT NULL,
    engine_number VARCHAR(50) DEFAULT NULL,
    make VARCHAR(100) NOT NULL,
    model VARCHAR(100) NOT NULL,
    year INT DEFAULT NULL,
    fuel_type ENUM('Petrol', 'Diesel', 'Electric', 'CNG', 'Hybrid') DEFAULT 'Petrol',
    vehicle_type VARCHAR(50) DEFAULT 'Motorcycle',
    color VARCHAR(50) DEFAULT NULL,
    odometer_reading INT DEFAULT 0,
    insurance_expiry DATE DEFAULT NULL,
    fitness_expiry DATE DEFAULT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
    INDEX idx_registration (registration_number),
    INDEX idx_chassis (chassis_number),
    INDEX idx_engine (engine_number)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ------------------------------------------------------
-- Table: job_cards
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS job_cards (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_number VARCHAR(50) UNIQUE NOT NULL,
    customer_id INT NOT NULL,
```

```
    vehicle_id INT NOT NULL,
    created_by INT NOT NULL,
    status ENUM('scheduled', 'listed', 'working', 'complete') DEFAULT 'listed',
    expected_delivery_date DATE DEFAULT NULL,
    description TEXT,
    advance_payment DECIMAL(10,2) DEFAULT 0.00,
    discount DECIMAL(10,2) DEFAULT 0.00,
    subtotal DECIMAL(10,2) DEFAULT 0.00,
    gst_amount DECIMAL(10,2) DEFAULT 0.00,
    final_total DECIMAL(10,2) DEFAULT 0.00,
    payment_status ENUM('pending', 'partial', 'paid') DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
    INDEX idx_status (status),
    INDEX idx_job_number (job_number),
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ------------------------------------------------------
-- Table: job_card_mechanics (supports multiple mechanics)
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS job_card_mechanics (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_card_id INT NOT NULL,
```

```
    mechanic_id INT NOT NULL,
    assigned_by INT NOT NULL,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_card_id) REFERENCES job_cards(id) ON DELETE CASCADE,
    FOREIGN KEY (mechanic_id) REFERENCES users(id),
    FOREIGN KEY (assigned_by) REFERENCES users(id),
    UNIQUE KEY unique_assignment (job_card_id, mechanic_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: job_card_items
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS job_card_items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_card_id INT NOT NULL,
    item_type ENUM('service', 'spare_part', 'outsourced') NOT NULL,
    item_name VARCHAR(255) NOT NULL,
    description TEXT,
    quantity INT DEFAULT 1,
    unit_price DECIMAL(10,2) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    is_customer_supplied BOOLEAN DEFAULT FALSE,
    inventory_id INT DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_card_id) REFERENCES job_cards(id) ON DELETE CASCADE,
    FOREIGN KEY (inventory_id) REFERENCES inventory(id) ON DELETE SET NULL,
    INDEX idx_item_type (item_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- ------------------------------------------------------
-- Table: job_card_photos
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS job_card_photos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_card_id INT NOT NULL,
    photo_url VARCHAR(500) NOT NULL,
    relative_path VARCHAR(500) NOT NULL,
    photo_type ENUM('before', 'after', 'issue') NOT NULL,
    description TEXT,
    uploaded_by INT NOT NULL,
    is_issue BOOLEAN DEFAULT FALSE,
    notified_to_customer BOOLEAN DEFAULT FALSE,
    customer_accepted BOOLEAN DEFAULT NULL,
    file_name VARCHAR(255) NOT NULL,
    file_size INT DEFAULT 0,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_card_id) REFERENCES job_cards(id) ON DELETE CASCADE,
    FOREIGN KEY (uploaded_by) REFERENCES users(id),
    INDEX idx_photo_type (photo_type),
    INDEX idx_is_issue (is_issue)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: inventory
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS inventory (
```

```
    id INT PRIMARY KEY AUTO_INCREMENT,
    part_name VARCHAR(255) NOT NULL,
    part_code VARCHAR(100) UNIQUE,
    category VARCHAR(100),
    supplier_id INT,
    current_stock INT DEFAULT 0,
    min_stock_level INT DEFAULT 5,
    unit VARCHAR(20) DEFAULT 'pcs',
    current_price DECIMAL(10,2) NOT NULL,
    barcode VARCHAR(100) DEFAULT NULL,
    location VARCHAR(100) DEFAULT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id) ON DELETE SET NULL,
    INDEX idx_barcode (barcode),
    INDEX idx_part_code (part_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- -----------------------------------------------------
-- Table: suppliers
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS suppliers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(255),
```

```
    address TEXT,
    gst_number VARCHAR(50),
    contact_person VARCHAR(255),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    INDEX idx_phone (phone)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ------------------------------------------------------
-- Table: purchase_orders
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS purchase_orders (
    id INT PRIMARY KEY AUTO_INCREMENT,
    po_number VARCHAR(50) UNIQUE NOT NULL,
    supplier_id INT NOT NULL,
    purchase_date DATE NOT NULL,
    total_amount DECIMAL(10,2) DEFAULT 0.00,
    discount DECIMAL(10,2) DEFAULT 0.00,
    gst_amount DECIMAL(10,2) DEFAULT 0.00,
    grand_total DECIMAL(10,2) DEFAULT 0.00,
    status ENUM('draft', 'ordered', 'received', 'cancelled') DEFAULT 'draft',
    notes TEXT,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id),
```

```
    FOREIGN KEY (created_by) REFERENCES users(id),
    INDEX idx_po_number (po_number),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: purchase_order_items
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS purchase_order_items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    purchase_order_id INT NOT NULL,
    inventory_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (purchase_order_id) REFERENCES purchase_orders(id) ON DELETE CASCADE,
    FOREIGN KEY (inventory_id) REFERENCES inventory(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: appointments
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS appointments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
```

```
    issues_description TEXT,
    status ENUM('pending', 'approved', 'rejected', 'completed') DEFAULT 'pending',
    approved_date DATE DEFAULT NULL,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
    INDEX idx_status (status),
    INDEX idx_appointment_date (appointment_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ------------------------------------------------------
-- Table: payments
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS payments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_card_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_type ENUM('cash', 'upi', 'bank', 'advance') NOT NULL,
    transaction_id VARCHAR(100) DEFAULT NULL,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    notes TEXT,
    received_by INT NOT NULL,
    FOREIGN KEY (job_card_id) REFERENCES job_cards(id) ON DELETE CASCADE,
    FOREIGN KEY (received_by) REFERENCES users(id),
    INDEX idx_payment_type (payment_type),
```

```
    INDEX idx_payment_date (payment_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ----------------------------------------------------
-- Table: attendance
-- ----------------------------------------------------
CREATE TABLE IF NOT EXISTS attendance (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    date DATE NOT NULL,
    check_in TIME NOT NULL,
    check_out TIME DEFAULT NULL,
    status ENUM('present', 'absent', 'half_day') DEFAULT 'present',
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    UNIQUE KEY unique_attendance (user_id, date),
    INDEX idx_date (date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ----------------------------------------------------
-- Table: leaves
-- ----------------------------------------------------
CREATE TABLE IF NOT EXISTS leaves (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
```

```
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    leave_type ENUM('sick', 'casual', 'emergency', 'other') NOT NULL,
    reason TEXT,
    status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending',
    approved_by INT DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (approved_by) REFERENCES users(id),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ------------------------------------------------------
-- Table: backup_logs
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS backup_logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    backup_name VARCHAR(255) NOT NULL,
    backup_path VARCHAR(500) NOT NULL,
    backup_size VARCHAR(50),
    type ENUM('manual', 'automatic') NOT NULL,
    created_by INT NOT NULL,
    status ENUM('success', 'failed') DEFAULT 'success',
    error_message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
```

```
    INDEX idx_type (type),
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: system_settings
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS system_settings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    setting_key VARCHAR(100) UNIQUE NOT NULL,
    setting_value TEXT,
    description TEXT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: auto_backup_settings
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS auto_backup_settings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    frequency ENUM('daily', 'weekly', 'monthly') DEFAULT 'daily',
    time TIME DEFAULT '02:00:00',
    last_backup TIMESTAMP NULL,
    is_active BOOLEAN DEFAULT TRUE,
    notification_phone VARCHAR(20),
    retention_days INT DEFAULT 30,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: module_visibility
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS module_visibility (
    id INT PRIMARY KEY AUTO_INCREMENT,
    role ENUM('owner', 'mechanic', 'customer') NOT NULL,
    module_name VARCHAR(100) NOT NULL,
    is_visible BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    UNIQUE KEY unique_role_module (role, module_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- Table: whatsapp_logs
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS whatsapp_logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    job_card_id INT DEFAULT NULL,
    customer_id INT NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    message_type VARCHAR(50) NOT NULL,
    message_text TEXT NOT NULL,
    status ENUM('sent', 'delivered', 'failed') DEFAULT 'sent',
```

```
    response_data TEXT,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (job_card_id) REFERENCES job_cards(id) ON DELETE SET NULL,
    INDEX idx_phone (phone_number),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ============================================
-- INITIAL DATA
-- ============================================

-- Insert default settings
INSERT INTO system_settings (setting_key, setting_value, description) VALUES
('app_name', 'Garage Management CRM', 'Application name'),
('currency', '₹', 'Currency symbol'),
('gst_rate', '18', 'Default GST rate'),
('country', 'India', 'Country'),
('timezone', 'Asia/Kolkata', 'Default timezone');

-- Insert default module visibility
INSERT INTO module_visibility (role, module_name, is_visible) VALUES
('owner', 'dashboard', 1),
('owner', 'job_cards', 1),
('owner', 'inventory', 1),
('owner', 'payments', 1),
('owner', 'customers', 1),
('owner', 'vehicles', 1),
```

('owner', 'reports', 1),

('owner', 'settings', 1),

('mechanic', 'dashboard', 1),

('mechanic', 'job_cards', 1),

('mechanic', 'inventory', 0),

('mechanic', 'payments', 0),

('mechanic', 'attendance', 1),

('customer', 'dashboard', 1),

('customer', 'appointments', 1),

('customer', 'vehicles', 1),

('customer', 'history', 1);

-- Insert default admin user (change password after first login)

INSERT INTO users (username, email, password_hash, phone, role, full_name, is_active) VALUES

('admin', 'admin@garage.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9999999999', 'super_admin', 'Super Admin', 1);

-- Default password: password

---

6. FOLDER STRUCTURE

6.1 Complete File Structure

text

garage-crm/

├── public/

│ ├── index.php

│ ├── .htaccess

│ └── assets/

│ ├── css/

```
|   |   ├── admin.css
|   |   ├── app.css
|   |   └── responsive.css
|   ├── js/
|   |   ├── app.js
|   |   ├── dashboard.js
|   |   ├── job-card.js
|   |   ├── inventory.js
|   |   ├── barcode-scanner.js
|   |   └── photo-upload.js
|   └── images/
|       ├── logo.png
|       └── favicon.ico
├── app/
|   ├── Http/
|   |   ├── Controllers/
|   |   |   ├── AuthController.php
|   |   |   ├── DashboardController.php
|   |   |   ├── JobCardController.php
|   |   |   ├── CustomerController.php
|   |   |   ├── VehicleController.php
|   |   |   ├── InventoryController.php
|   |   |   ├── PaymentController.php
|   |   |   ├── AppointmentController.php
|   |   |   ├── ReportController.php
|   |   |   └── BackupController.php
|   |   ├── Middleware/
```

```
|   |   |   ├── RoleMiddleware.php
|   |   |   ├── AuthMiddleware.php
|   |   |   └── CorsMiddleware.php
|   |   └── Requests/
|   |       ├── JobCardRequest.php
|   |       ├── CustomerRequest.php
|   |       └── InventoryRequest.php
|   ├── Models/
|   |   ├── User.php
|   |   ├── Customer.php
|   |   ├── Vehicle.php
|   |   ├── JobCard.php
|   |   ├── JobCardItem.php
|   |   ├── JobCardPhoto.php
|   |   ├── Inventory.php
|   |   ├── Supplier.php
|   |   ├── PurchaseOrder.php
|   |   ├── Appointment.php
|   |   ├── Payment.php
|   |   ├── Attendance.php
|   |   └── BackupLog.php
|   └── Services/
|       ├── WhatsAppService.php
|       ├── BackupService.php
|       ├── ReportService.php
|       └── PhotoService.php
├── database/
```

```
|   ├── migrations/
|   |   ├── 2026_01_01_000001_create_users_table.php
|   |   ├── 2026_01_01_000002_create_customers_table.php
|   |   ├── 2026_01_01_000003_create_vehicles_table.php
|   |   ├── 2026_01_01_000004_create_job_cards_table.php
|   |   └── [other migration files]
|   └── seeds/
|       ├── DatabaseSeeder.php
|       ├── UserSeeder.php
|       └── SettingSeeder.php
├── resources/
|   ├── views/
|   |   ├── layouts/
|   |   |   ├── admin.blade.php
|   |   |   └── app.blade.php
|   |   ├── dashboard/
|   |   |   ├── owner.blade.php
|   |   |   ├── mechanic.blade.php
|   |   |   └── customer.blade.php
|   |   ├── job-cards/
|   |   |   ├── index.blade.php
|   |   |   ├── create.blade.php
|   |   |   └── show.blade.php
|   |   └── [other view files]
|   └── lang/
|       └── en/
|           ├── messages.php
```

```
|       └── validation.php
├── routes/
|   ├── web.php
|   └── api.php
├── storage/
|   ├── app/
|   |   └── public/
|   |       └── uploads/
|   |           └── job_cards/
|   |               └── {customer_name}/
|   |                   └── {vehicle_registration}/
|   |                       └── {DDMMYYYY}/
|   |                           └── JC{job_id}/
|   |                               ├── before/
|   |                               ├── after/
|   |                               └── issues/
|   ├── backups/
|   |   └── [backup files - .zip]
|   ├── logs/
|   |   └── laravel.log
|   └── framework/
|       ├── cache/
|       ├── sessions/
|       └── views/
├── config/
|   ├── app.php
|   ├── database.php
```

```
|   ├── services.php
|   └── backup.php
├── .env
├── .env.example
├── composer.json
├── package.json
├── artisan
└── README.md
```

6.2 Photo Storage Structure (Detailed)

text

```
/uploads/
└── job_cards/
    └── {customer_name}/            # Sanitized customer name
        └── {vehicle_registration}/   # Vehicle registration number
            └── {DDMMYYYY}/         # Date of job card creation
                └── JC{job_id}/      # Job Card ID
                    ├── before/       # Before service photos
                    |   ├── before_1638123456789_abc123.jpg
                    |   └── before_1638123456789_def456.jpg
                    ├── after/        # After service photos
                    |   └── after_1638123456789_ghi789.jpg
                    └── issues/       # Issue/defect photos
                        ├── brake_issue_1638123456789_jkl012.jpg
                        └── oil_leak_1638123456789_mno345.jpg
```

Folder Structure Examples:

text

```
Example 1: Raj Sharma's vehicle GJ01AB1234, Job Card #42
```

/uploads/job_cards/Raj_Sharma/GJ01AB1234/26072026/JC42/

├── before/

│ └── before_1638123456_abc.jpg

├── after/

│ └── after_1638123457_def.jpg

└── issues/

└── brake_issue_1638123458_ghi.jpg

Example 2: Priya Patel's vehicle MH02CD5678, Job Card #87

/uploads/job_cards/Priya_Patel/MH02CD5678/28072026/JC87/

├── before/

│ └── before_1638124567_jkl.jpg

└── issues/

└── engine_issue_1638124568_mno.jpg

---

7. MODULE SPECIFICATIONS

7.1 Dashboard Module

Owner Dashboard

Statistics Cards:

- Total Job Cards (with breakdown)
- Total Customers
- Active Mechanics
- Pending Appointments

Financial Overview:

- Total Revenue
- Total Income
- Estimated Profit
- Total Outstanding

- Total Expenses

Payment Breakdown:

- Cash Payments
- UPI Payments
- Bank Transfers

Recent Activity:

- New job cards
- Status updates
- Payment receipts

Mechanic Dashboard

- Assigned job cards
- Today's schedule
- Attendance status
- Pending tasks

Customer Dashboard

- My vehicles
- Active service jobs
- Service history
- Appointment status

7.2 Job Card Management

Create Job Card Flow

Step 1: Select Customer

- Search existing customer
- Create new customer (if not found)
- Auto-fill customer details

Step 2: Select Vehicle

- Show all vehicles for selected customer
- Add new vehicle

- Auto-fill vehicle details (registration, make, model, etc.)

Step 3: Add Services/Items

- Add service charges
- Add spare parts (with inventory integration)
- Mark customer-supplied parts
- Apply GST (Global or Item-wise)

Step 4: Assign Mechanics

- Auto-assign if created by mechanic
- Multi-select mechanics (owner only)
- View assigned mechanics

Step 5: Upload Photos

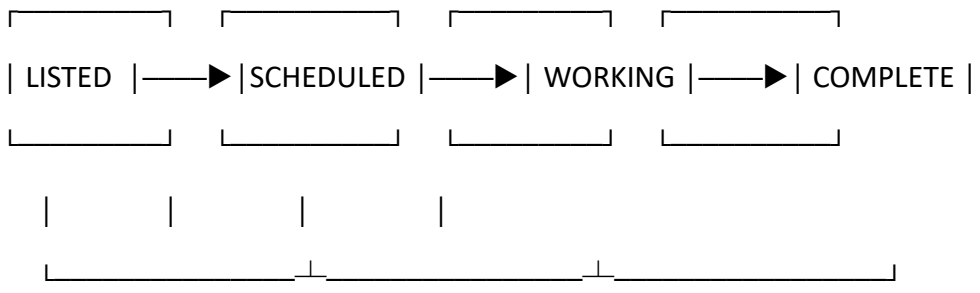
- Before photos (required)
- After photos (optional)
- Issue photos (for customer notification)

Step 6: Set Delivery Date

- Expected delivery date
- Priority (normal/urgent)

Job Card Status Flow

text



Status Descriptions:

- **Listed**: Job card created, pending scheduling

- **Scheduled**: Delivery date set, parts arranged
- **Working**: Mechanic actively working on vehicle
- **Complete**: Work finished, ready for pickup

7.3 Customer & Vehicle Management

Customer Management

- Add/Edit/Delete customers
- Search customers (name, phone, email)
- View customer service history
- Manage multiple vehicles
- Send WhatsApp notifications

Vehicle Management

- Register new vehicles
- Track service history per vehicle
- Store registration number (primary identifier)
- Store Chassis Number (VIN) and Engine Number
- Service due reminders

7.4 Inventory Management

Features

- **Add New Items:** Name, code, category, price, stock
- **Barcode Scanner:** Scan QR/barcode to find/add items
- **Price Management:**
    - Current price stored in inventory
    - Price override per invoice (doesn't affect history)
    - Previous invoices keep original prices
- **Stock Tracking:** Real-time updates with purchase orders
- **Low Stock Alerts:** Notify when below minimum level

7.5 Appointment System

Flow

1. Customer creates appointment
2. Owner receives notification
3. Owner approves/rejects
4. Customer receives WhatsApp confirmation
5. Status updates throughout

7.6 Payment Management

Payment Types

- Cash
- UPI
- Bank Transfer
- Advance Payment

Features

- Record payments against job cards
- Track outstanding amounts
- Adjust advance payments
- Generate receipts

7.7 Attendance & Leave Management

Attendance

- Clock in/out for mechanics
- Daily attendance tracking
- Report generation

Leaves

- Apply for leave
- Approve/Reject (owner only)
- Leave balance tracking

7.8 Reports Module

Available Reports

- Financial Summary

- Job Cards Report
- Inventory Report
- Mechanic Performance
- Customer History
- Revenue Report

Export Options

- PDF
- Excel (CSV)
- Print

---

8. API ENDPOINTS

8.1 Authentication APIs

text

POST /api/auth/login - User login

POST /api/auth/logout - User logout

POST /api/auth/register - User registration

POST /api/auth/refresh - Refresh token

GET /api/auth/me - Get current user

POST /api/auth/forgot-password - Forgot password

POST /api/auth/reset-password - Reset password

8.2 Dashboard APIs

text

GET /api/dashboard/stats - Get statistics

GET /api/dashboard/financials - Get financial data

GET /api/dashboard/recent - Get recent activities

8.3 Job Card APIs

text

GET /api/job-cards - List all job cards

POST /api/job-cards - Create new job card

GET /api/job-cards/{id} - Get job card details

PUT /api/job-cards/{id} - Update job card

DELETE /api/job-cards/{id} - Delete job card

POST /api/job-cards/{id}/status - Update status

POST /api/job-cards/{id}/photos - Upload photos

DELETE /api/job-cards/{id}/photos/{photoId} - Delete photo

POST /api/job-cards/{id}/mechanics - Assign mechanic

DELETE /api/job-cards/{id}/mechanics/{mechanicId} - Remove mechanic

POST /api/job-cards/{id}/items - Add item to job card

DELETE /api/job-cards/{id}/items/{itemId} - Remove item

8.4 Customer APIs

text

GET /api/customers - List all customers

POST /api/customers - Create customer

GET /api/customers/{id} - Get customer details

PUT /api/customers/{id} - Update customer

DELETE /api/customers/{id} - Delete customer

GET /api/customers/search - Search customers

GET /api/customers/{id}/vehicles - Get customer's vehicles

GET /api/customers/{id}/history - Get service history

8.5 Vehicle APIs

text

GET /api/vehicles - List all vehicles

POST /api/vehicles - Create vehicle

GET /api/vehicles/{id} - Get vehicle details

PUT /api/vehicles/{id} - Update vehicle

DELETE /api/vehicles/{id} - Delete vehicle

GET /api/vehicles/search - Search vehicles

GET /api/vehicles/customer/{customerId} - Get vehicles by customer

8.6 Inventory APIs

text

GET /api/inventory - List all inventory items

POST /api/inventory - Create inventory item

GET /api/inventory/{id} - Get inventory item

PUT /api/inventory/{id} - Update inventory item

DELETE /api/inventory/{id} - Delete inventory item

GET /api/inventory/barcode/{barcode} - Find by barcode

GET /api/inventory/low-stock - Get low stock items

POST /api/inventory/{id}/adjust-stock - Adjust stock

8.7 Supplier APIs

text

GET /api/suppliers - List all suppliers

POST /api/suppliers - Create supplier

GET /api/suppliers/{id} - Get supplier details

PUT /api/suppliers/{id} - Update supplier

DELETE /api/suppliers/{id} - Delete supplier

8.8 Purchase Order APIs

text

GET /api/purchase-orders - List all purchase orders

POST /api/purchase-orders - Create purchase order

GET /api/purchase-orders/{id} - Get purchase order details

PUT /api/purchase-orders/{id} - Update purchase order

DELETE /api/purchase-orders/{id} - Delete purchase order

POST /api/purchase-orders/{id}/receive - Mark as received

8.9 Appointment APIs

text

GET /api/appointments - List all appointments

POST /api/appointments - Create appointment

GET /api/appointments/{id} - Get appointment details

PUT /api/appointments/{id}/status - Update status

GET /api/appointments/customer - Get customer appointments

GET /api/appointments/pending - Get pending appointments

8.10 Payment APIs

text

GET /api/payments - List all payments

POST /api/payments - Create payment

GET /api/payments/{id} - Get payment details

GET /api/payments/job-card/{jobCardId} - Get payments by job card

GET /api/payments/outstanding - Get outstanding payments

8.11 Report APIs

text

GET /api/reports/financial - Financial report

GET /api/reports/job-cards - Job cards report

GET /api/reports/inventory - Inventory report

GET /api/reports/mechanics - Mechanics report

GET /api/reports/export/{type} - Export report

8.12 Backup APIs (Super Admin Only)

text

POST /api/backup/create - Create backup

POST /api/backup/restore - Restore backup

GET /api/backup/list - List backups

DELETE /api/backup/{id} - Delete backup

POST /api/backup/schedule - Schedule automatic backup

GET /api/backup/settings - Get backup settings

8.13 Settings APIs

text

GET /api/settings - Get all settings

PUT /api/settings - Update settings

GET /api/settings/modules - Get module visibility

PUT /api/settings/modules - Update module visibility

8.14 User Management APIs

text

GET /api/users - List all users

POST /api/users - Create user

GET /api/users/{id} - Get user details

PUT /api/users/{id} - Update user

DELETE /api/users/{id} - Delete user

POST /api/users/{id}/role - Update user role

---

9. WHATSAPP INTEGRATION

9.1 Integration Setup

WhatsApp Business API Configuration

php

```
// config/services.php
'whatsapp' => [
    'api_url' => env('WHATSAPP_API_URL', 'https://api.whatsapp.com/v1'),
    'api_key' => env('WHATSAPP_API_KEY'),
    'phone_number' => env('WHATSAPP_PHONE_NUMBER'),
],
```

Environment Variables

env

WHATSAPP_API_URL=https://api.whatsapp.com/v1

WHATSAPP_API_KEY=your_whatsapp_api_key

WHATSAPP_PHONE_NUMBER=918XXXXXXXXX

9.2 Automated Notifications

When to Send Notifications

Event	Message Type	Recipient
Job Card Created	Creation Confirmation	Customer
Status Changed	Status Update	Customer
Issue Photo Uploaded	Issue Alert	Customer
Appointment Created	Pending Confirmation	Customer
Appointment Approved	Confirmation	Customer
Job Completed	Ready for Pickup	Customer
Payment Reminder	Reminder	Customer
Backup Completed	Success Notification	Owner
Backup Failed	Error Notification	Owner

9.3 Message Templates

Job Card Creation

text

🛠️ Your vehicle has been registered for service!

Job Card: {job_number}

Vehicle: {registration_number}

Status: Listed

Thank you for choosing our services.

Status Update

text

📱 Status Update for your vehicle {registration_number}

Job Card: {job_number}

New Status: {status}

Expected Delivery: {delivery_date}

We'll keep you updated! 🚗

Issue Found

text

⚠️ Issue Found on Your Vehicle {registration_number}

We found an issue with: {part_name}

Please login to accept or reject the repair:

{app_url}/customer/login

Ready for Pickup

text

✅ Your vehicle {registration_number} is ready for pickup! 🚗

Job Card: {job_number}

Total Amount: ₹{amount}

Payment: {payment_status}

Thank you for choosing us!

---

10. BACKUP & RESTORE SYSTEM

10.1 Backup Service

Manual Backup

- **Trigger:** Super Admin only
- **Content:** Complete database + all uploaded photos
- **Output:** Single ZIP file with timestamp

- **Storage:** /storage/backups/

Automatic Backup

- **Schedule:** Daily/Weekly/Monthly (configurable)
- **Time:** Configurable (default: 2:00 AM)
- **Retention:** Configurable (default: 30 days)
- **Notification:** WhatsApp alert to owner on completion/failure

10.2 Backup Files Structure

text

```
backup_20260726_020000.zip
├── database.sql        # Complete database dump
└── uploads/            # All uploaded photos
    └── job_cards/
        ├── Raj_Sharma/
        |   └── GJ01AB1234/
        |       └── 26072026/
        |           └── JC42/
        |               ├── before/
        |               ├── after/
        |               └── issues/
        └── [other customers...]
```

10.3 Restore Process

1. Upload backup ZIP file
2. System extracts files
3. Restores database
4. Copies photos to storage
5. Verifies integrity
6. Notifies on completion

10.4 Cron Job for Automatic Backup

bash

Schedule: Every hour to check if backup is needed

* * * * * php /path/to/garage-crm/artisan schedule:run >> /dev/null 2>&1

Laravel Schedule (app/Console/Kernel.php)

protected function schedule(Schedule $schedule)

{

$schedule->command('backup:auto')->hourly();

$schedule->command('backup:cleanup')->daily();

}

---

11. SECURITY IMPLEMENTATION

11.1 Authentication

- JWT (JSON Web Token) based authentication
- Session timeout after 2 hours of inactivity
- Password hashed using bcrypt
- Rate limiting on login attempts (5 attempts/hour)

11.2 Authorization

- Role-based access control (RBAC)
- Module-level permissions
- Resource-level permissions (own data only)

11.3 Data Protection

- SQL injection prevention (Eloquent ORM)
- XSS prevention (auto-escaping)
- CSRF protection (token-based)
- File upload validation (size, type, content)

11.4 Secure Headers

apache

Header set X-Content-Type-Options "nosniff"

Header set X-Frame-Options "DENY"

Header set X-XSS-Protection "1; mode=block"

Header set Referrer-Policy "strict-origin-when-cross-origin"

11.5 Password Policy

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character

---

12. LOCAL DEVELOPMENT SETUP

12.1 Prerequisites

Install the following on your laptop:

1. **PHP 8.0+**
    - Download: https://www.php.net/downloads
    - Required extensions: BCMath, Ctype, Fileinfo, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, CURL, GD, Exif
2. **MySQL 5.7+**
    - Download: https://www.mysql.com/downloads
    - Or use XAMPP/WAMP/Laragon
3. **Composer**
    - Download: https://getcomposer.org/download
4. **Node.js & NPM**
    - Download: https://nodejs.org/en/download
5. **Git**
    - Download: https://git-scm.com/downloads

12.2 Step-by-Step Setup

Step 1: Clone Repository

bash

Open terminal/command prompt

cd C:/xampp/htdocs *# or your web server directory*

git clone https://your-repository-url/garage-crm.git

cd garage-crm

Step 2: Install Dependencies

bash

Install PHP dependencies

composer install

Install Node dependencies

npm install

Step 3: Environment Configuration

bash

Copy environment file

cp .env.example .env

Edit .env file with your local settings

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=garage_crm

DB_USERNAME=root

DB_PASSWORD=your_mysql_password

Step 4: Generate Application Key

bash

php artisan key:generate

Step 5: Create Database

sql

-- In MySQL

CREATE DATABASE garage_crm;

Step 6: Run Migrations & Seeders

bash

php artisan migrate

php artisan db:seed

Step 7: Create Storage Link

bash

php artisan storage:link

Step 8: Set File Permissions (Linux/Mac)

bash

chmod -R 775 storage

chmod -R 775 bootstrap/cache

chmod -R 775 public/uploads

Step 9: Build Frontend Assets

bash

npm run build

Or for development

npm run dev

Step 10: Start Development Server

bash

PHP built-in server

php artisan serve

For hot reloading (if using Laravel Mix)

npm run watch

12.3 Access the Application

- **Admin Login:** http://localhost:8000
- **Default Admin Credentials:**
    - Email: admin@garage.com
    - Password: password

12.4 Local Development Tips

Enable Debug Mode

env

APP_ENV=local

APP_DEBUG=true

Use SQLite for Testing (Optional)

env

DB_CONNECTION=sqlite

Disable CORS for Local Development

php

// In config/cors.php

'allowed_origins' => ['*'],

---

13. HOSTINGER DEPLOYMENT GUIDE

13.1 Hostinger Cloud Startup Plan Details

- **Storage:** 50GB SSD
- **Bandwidth:** 1TB
- **CPU:** 2 vCPU
- **RAM:** 2GB
- **PHP Version:** 8.0+
- **MySQL:** 5.7+
- **Free SSL:** Included
- **Domain:** Free domain included

13.2 Pre-Deployment Checklist

- All features tested locally
- Database migrations working
- All dependencies installed
- Storage permissions correct
- Environment variables configured
- SSL certificate ready

13.3 Deployment Steps

Step 1: Set Up Hosting Account

1. Login to Hostinger hPanel
2. Go to Websites → Add Website
3. Enter your domain name
4. Choose "Cloud Startup Plan"
5. Complete the setup

Step 2: Prepare Files

bash

Create deployment package

composer install --no-dev --optimize-autoloader

npm run production

Create ZIP file

zip -r garage-crm.zip . -x "node_modules/*" ".git/*" "storage/*"

Step 3: Upload Files

bash

Using FTP or Hostinger File Manager

Upload garage-crm.zip

Extract to public_html or your domain folder

Step 4: Configure Database

bash

In Hostinger hPanel → MySQL

Create database: garage_crm

Create user: garage_user

Generate strong password

Grant all privileges

Step 5: Configure Environment

bash

Copy .env.example to .env

Update with production settings:

APP_ENV=production

APP_DEBUG=false

APP_URL=https://yourdomain.com

DB_DATABASE=garage_crm

DB_USERNAME=garage_user

DB_PASSWORD=your_strong_password

WHATSAPP_API_KEY=your_whatsapp_key

WHATSAPP_PHONE_NUMBER=your_business_phone

Step 6: Set File Permissions

bash

chmod -R 775 storage

chmod -R 775 bootstrap/cache

chmod -R 775 public/uploads

Step 7: Run Migrations

bash

php artisan migrate --force

php artisan db:seed --force

Step 8: Setup Cron Jobs

bash

In Hostinger hPanel → Advanced → Cron Jobs

Add this command:

* * * * * php /home/yourusername/garage-crm/artisan schedule:run

Step 9: Configure .htaccess

apache

public/.htaccess

<IfModule mod_rewrite.c>

    RewriteEngine On

    RewriteCond %{REQUEST_FILENAME} !-d

    RewriteCond %{REQUEST_FILENAME} !-f

    RewriteRule ^ index.php [L]

</IfModule>

Step 10: Test the Application

1. Visit https://yourdomain.com
2. Login with admin credentials
3. Test all modules
4. Check WhatsApp integration
5. Test photo upload
6. Verify database connection

13.4 Post-Deployment Tasks

1. **Enable SSL** (if not already enabled)
2. **Set up Google Analytics**
3. **Configure email service**
4. **Test WhatsApp notifications**
5. **Set up automatic backups**

6. **Create admin user accounts**
7. **Train the owner on usage**

13.5 Performance Optimization for Hostinger

PHP Configuration

ini

upload_max_filesize = 50M

post_max_size = 50M

max_execution_time = 300

memory_limit = 256M

MySQL Optimization

sql

-- In Hostinger hPanel → MySQL

-- Increase buffer pool size (if allowed)

SET GLOBAL innodb_buffer_pool_size = 128M;

Cache Enablement

bash

Enable Laravel caches

php artisan config:cache

php artisan route:cache

php artisan view:cache

---

14. TESTING CHECKLIST

14.1 Functionality Testing

Authentication

- Login with correct credentials
- Login with incorrect credentials
- Password reset
- Session timeout

- Logout
- Remember me functionality

Dashboard

- Owner dashboard loads correctly
- Mechanic dashboard loads correctly
- Customer dashboard loads correctly
- Statistics display correctly
- Financial data is accurate
- Recent activities are shown

Job Cards

- Create job card with all fields
- Search customers
- Add multiple items to job card
- Upload photos (before/after/issue)
- Update job card status
- Assign mechanic(s)
- Auto-assign when mechanic creates
- Multi-mechanic assignment by owner
- View job card details
- Edit job card
- Delete job card
- Print job card

Customer Management

- Add new customer
- Update customer details
- Delete customer
- Add multiple vehicles
- Search customers

- Customer login
- Customer appointment creation
- Customer view own job cards
- Customer profile update

Inventory

- Add new inventory item
- Update stock levels
- Barcode scanning
- Low stock alerts
- Price update without affecting history
- Search inventory
- Delete inventory item

Payments

- Record payment
- Advance payment tracking
- Outstanding amount calculation
- Payment reconciliation
- GST calculation
- Payment receipts
- Export payment reports

WhatsApp

- Status update notifications
- Issue photo notifications
- Appointment confirmations
- Payment reminders
- Backup notifications
- Message delivery status

Backup

- Manual backup creation
- Automatic backup scheduling
- Backup restoration
- Backup file download
- Backup notification
- Backup integrity verification

Reports

- Financial report generation
- Job cards report
- Inventory report
- Mechanic performance report
- Export to PDF
- Export to Excel

14.2 Performance Testing

- Page load time < 3 seconds
- Image upload speed (< 10 seconds for 5MB)
- Search functionality speed (< 2 seconds)
- Concurrent user sessions (10+ users)
- Database query optimization
- Memory usage under 256MB

14.3 Security Testing

- SQL injection prevention
- XSS prevention
- CSRF protection
- Session security
- File upload validation
- Authentication bypass attempts
- Password security

- API endpoint protection

14.4 Mobile Responsiveness

- Desktop view (1920x1080)
- Tablet view (768x1024)
- Mobile view (375x812)
- Touch interface
- Camera integration
- Mobile form submission
- Swipe gestures

14.5 Browser Compatibility

- Google Chrome (Latest)
- Mozilla Firefox (Latest)
- Safari (Latest)
- Microsoft Edge (Latest)
- Mobile browsers (Chrome, Safari)

---

15. MAINTENANCE & MONITORING

15.1 Daily Maintenance Tasks

- Check backup logs
- Monitor storage usage
- Review error logs
- Check WhatsApp message delivery
- Verify automated tasks

15.2 Weekly Maintenance Tasks

- Review system performance
- Check low inventory items
- Review outstanding payments
- Update security patches

- Database optimization

15.3 Monthly Maintenance Tasks

- Full security audit
- Performance optimization
- Clean up old backups
- Update dependencies
- Review user accounts

15.4 Monitoring URLs

text

```
GET /health          - System health check
GET /api/status      - API status
GET /logs            - Error logs (admin only)
GET /backup/status   - Backup status
```

15.5 Error Monitoring

Log File Location

text

```
storage/logs/laravel.log
```

Common Errors & Solutions

Error	Solution
500 Internal Server Error	Check permissions, memory limit
Database Connection Failed	Verify database credentials
404 Not Found	Check routes, file permissions
Upload Failed	Check upload limits, permissions
WhatsApp API Error	Verify API key, phone number

---

16. TROUBLESHOOTING GUIDE

16.1 Common Issues and Solutions

Issue: 500 Internal Server Error

text

Solution:

1. Check file permissions (storage, bootstrap/cache)

2. Increase PHP memory limit

3. Check .env file for syntax errors

4. Run composer dump-autoload

5. Check error logs

Issue: Database Connection Failed

text

Solution:

1. Verify database credentials in .env

2. Check MySQL is running

3. Create database if missing

4. Grant privileges to user

5. Check host (localhost vs 127.0.0.1)

Issue: Photos Not Uploading

text

Solution:

1. Check folder permissions (storage/app/public/uploads)

2. Create uploads directory

3. Check PHP upload limits (size, max)

4. Verify file type is allowed

5. Run php artisan storage:link

Issue: WhatsApp Messages Not Sending

text

Solution:

1. Verify API key

2. Check phone number format

3. Check internet connection

4. Verify WhatsApp Business API subscription

5. Check error logs

Issue: Backup Fails

text

Solution:

1. Check disk space

2. Verify MySQL credentials

3. Check backup folder permissions

4. Verify storage permissions

5. Check max execution time

Issue: Login Not Working

text

Solution:

1. Clear browser cache

2. Clear session: php artisan session:clear

3. Check user exists in database

4. Verify password hash

5. Check user is active (is_active = 1)

Issue: Job Card Not Creating

text

Solution:

1. Check all required fields

2. Verify customer exists

3. Verify vehicle exists

4. Check database constraints

5. Check validation rules

16.2 Maintenance Commands

bash

Clear all caches

php artisan cache:clear

php artisan config:clear

php artisan route:clear

php artisan view:clear

Optimize for production

php artisan config:cache

php artisan route:cache

php artisan view:cache

Run migrations

php artisan migrate

Run seeders

php artisan db:seed

Create storage link

php artisan storage:link

List all routes

php artisan route:list

Check application health

php artisan health:check

16.3 Support Resources

- **Documentation:** This document
- **GitHub Issues:** https://github.com/your-repo/issues
- **Email Support:** support@yourdomain.com
- **Emergency Contact:** [Developer Phone Number]

APPENDICES

Appendix A: Default Admin Credentials

Field	Value
Email	admin@garage.com
Password	password
Username	admin
Role	super_admin

Appendix B: Environment Variables Reference

env

Application

APP_NAME="Garage Management CRM"

APP_ENV=production

APP_DEBUG=false

APP_URL=https://yourdomain.com

Database

DB_CONNECTION=mysql

DB_HOST=localhost

DB_PORT=3306

DB_DATABASE=garage_crm

DB_USERNAME=garage_user

DB_PASSWORD=your_password

WhatsApp

WHATSAPP_API_URL=https://api.whatsapp.com/v1

WHATSAPP_API_KEY=your_api_key

WHATSAPP_PHONE_NUMBER=918XXXXXXXXX

Backup

BACKUP_PATH=/home/yourusername/backups

BACKUP_RETENTION_DAYS=30

Email

MAIL_MAILER=smtp

MAIL_HOST=smtp.gmail.com

MAIL_PORT=587

MAIL_USERNAME=your_email@gmail.com

MAIL_PASSWORD=your_app_password

MAIL_ENCRYPTION=tls

MAIL_FROM_ADDRESS="noreply@yourdomain.com"

MAIL_FROM_NAME="Garage Management CRM"

Appendix C: Useful Commands

bash

Development

php artisan serve *# Start development server*

```
npm run dev                    # Compile assets for development
npm run watch                  # Watch for changes

# Production
composer install --no-dev      # Install production dependencies
npm run production             # Compile assets for production
php artisan optimize           # Optimize application

# Database
php artisan migrate            # Run migrations
php artisan migrate:fresh      # Reset and run migrations
php artisan db:seed            # Run seeders

# Utilities
php artisan make:model ModelName     # Create model
php artisan make:controller ControllerName  # Create controller
php artisan make:migration migration_name  # Create migration
php artisan tinker             # Interactive shell
```

Appendix D: Project Timeline

Phase	Duration	Tasks
Phase 1	2 weeks	Database setup, Authentication
Phase 2	2 weeks	Job Cards, Customer Management
Phase 3	2 weeks	Inventory, Payments
Phase 4	2 weeks	WhatsApp Integration, Reports

Phase	Duration	Tasks
Phase 5	2 weeks	Backup System, Testing
Phase 6	2 weeks	Deployment, Training
Total	**12 weeks**	**Full System**

RELEASE NOTES

Version 1.0 - July 2026

Features:

- ✅ Complete job card management
- ✅ Customer and vehicle management
- ✅ Inventory management with barcode
- ✅ WhatsApp integration
- ✅ Automatic backup system
- ✅ Role-based dashboards
- ✅ Payment tracking
- ✅ Appointment system
- ✅ Multi-mechanic assignment

Technical Improvements:

- ✅ Optimized database queries
- ✅ Mobile responsive design
- ✅ Security enhancements
- ✅ Error handling
- ✅ Performance optimization

SUPPORT & CONTACT

Developer: [Your Name]
Email: [developer@email.com]
GitHub: [your-github-url]
Project: [project-url]

Owner Support:
Email: [owner-support@garage.com]
Phone: [support-phone-number]
Website: [support-website]

Documentation Version: 1.0
Last Updated: July 26, 2026
Document Prepared For: Google Antigravity Development Environment
Target Deployment: Hostinger Cloud Startup Plan

End of Document